摸鱼也升职游戏软件
软件设计说明书

版本号：V1.0

1 软件概述

竖屏职场模拟放置小游戏。玩家扮演一名从实习生干起的打工人，在工作和摸鱼两种状态间切换，一边赚工资攒经验冲职级，一边盯着绩效、心情、灵感三条状态轴别让任何一条崩掉。上班产出绩效但掉心情，摸鱼回心情涨灵感但绩效往下掉，玩家得自己找平衡。中途随机弹职场事件卡，退休后还能投胎重开继承一部分加成。

数据全部本地存储，不连服务器，靠激励视频变现。团结引擎C#开发，核心模拟逻辑是纯C#、不依赖Unity API，方便单独跑测试。

2 运行环境

项	要求
操作系统	Android 8.0及以上 / iOS 12.0及以上
运行平台	微信小游戏、抖音小游戏、移动端
游戏引擎	Unity 2021.3 LTS（团结引擎1.10.3）
编程语言	C#
运行内存	2GB以上
存储空间	100MB以上
屏幕分辨率	720x1280以上，竖屏

3 工程结构

运行时代码在 `Assets/_Game/Scripts/Runtime/`，按职责分目录：

```
Runtime/
├─ Boot/
```

```
│ ├─ BootRunner.cs
│ ├─ BootSequence.cs
│ └─ BootServices.cs
├─ Game/
│ ├─ WorkerData.cs
│ ├─ WorkerSim.cs  C#
│ ├─ WorkerMainHost.cs MonoBehaviour
│ ├─ WorkerEventFlow.cs
│ ├─ WorkerOffline.cs
│ ├─ WorkerSaveData.cs
│ ├─ WorkerGacha.cs
│ ├─ WorkerCareer.cs
│ └─ WorkerRankGate.cs
├─ Iaa/
│ └─ IaaGate.cs
├─ Audio/
│ └─ G11AudioCatalog.cs
└─ UI/
├─ HudView.cs
└─ EventCardView.cs
```

4 核心数值常量

WorkerData 里把所有平衡数值写成常量，运行时不改。下面是实际值。

基础经济：

常量	值	含义
BaseWagePerSec	1.0	实习生基础每秒工资
ExpPerSecOnline	1.0	在线每秒经验
CoinCap	100000000	工资上限1亿
ExpCap	1000000	经验上限100万
MaxOfflineHours	12	离线收益最多算12小时
WeekendOfflineMult	1.2	周末离线收益倍率

五条职级的系数，工资按基础值乘这个系数：

职级	实习生	专员	主管	经理	CEO
RankCoef	1.0	1.6	2.6	4.2	7.0

5 三条状态轴

模拟核心维护三条轴，取值都在0到100：

轴	字段	工作时变化	摸鱼时变化
绩效	Kpi	+WorkKpiPerSec	SlackKpiPerSec（负值）

心情	Mood	WorkMoodPerSec（负值）	+SlackMoodPerSec
灵感	Insp	0	+SlackInspPerSec

初始值 InitKpi、InitMood、InitInsp 都定在中间偏上，开局不会立刻崩。

轴的上下限：

常量	值	含义
AxisMin	0	轴下限
AxisMax	100	轴上限
MoodStrikeLine	10	心情跌破触发送茶/怠工事件
MoodRecoverTo	40	事件后心情恢复到的位置
KpiWarnLine	90	绩效过载警告线
RarePoolInspGate	60	稀有事件需要灵感达到

6 岗位软帽公式

工资不是线性无限涨，职级越高边际越收。岗位产出走对数软帽，核心代码逻辑：

```
//             CEO
float WagePerSec(int rankIndex, int pptLevels, int ngLoop)
{
float rank = RankCoef[Clamp(rankIndex)];
float skill = SkillMult(pptLevels, WageMultBonus);
float inherit = 1f + InheritWageMultPerLoop * Max(0, ngLoop);
return BaseWagePerSec * rank * skill * inherit;
}
```

四因子连乘：基础工资 × 职级系数 × PPT技能倍率 × 投胎继承倍率。投胎继承每多一周目工资加 InheritWageMultPerLoop。

7 技能系统

四个技能分支，每个分支3级：

SkillBranch	含义	升级效果
Ppt	做PPT	加工资倍率
Report	写汇报	加绩效获取
SlackArt	摸鱼艺术	摸鱼时心情倍率提升、L3减少绩效损失
ManageUp	向上管理	加离线收益

升级花经验，各级费用在 SkillUpgradeExpCost 数组。技能加成在 WageMultBonus、KpiGainBonus、SlackMoodMult、OfflineBonus 几张表里，按已学等级查表：

```
static float SkillMult(int levelsLearned, float[] bonusTable)
{
if (levelsLearned <= 0) return 1f;
```

```
return 1f + bonusTable[ClampIndex(levelsLearned - 1, bonusTable.Length)];
}
```

摸鱼艺术学到L3后，摸鱼时绩效掉速乘 SlackLossMultAtL3，比没学时慢。

8 工作与摸鱼

玩家点切换按钮在两种状态间切，WorkerSim.Working 记当前状态。

工作时：涨工资、涨经验、涨绩效，心情往下掉。摸鱼时：不产工资，绩效慢慢掉，心情和灵感往上涨。

状态轴的每秒变化由 WorkerData 的纯函数算，不写死在模拟里，调数值只动 WorkerData：

```
float KpiPerSec(bool working, int reportLv, int slackLv)
{
if (working) return WorkKpiPerSec + SkillBonus(reportLv, KpiGainBonus);
float lossMult = slackLv >= SkillLevelCount ? SlackLossMultAtL3 : 1f;
return SlackKpiPerSec * lossMult;
}
```

9 职场事件

事件由 WorkerEventFlow 按触发条件判定，WorkerData 里冻结了15条事件，每条是一个EventDef。触发类型 TriggerKind 有：Scripted（脚本）、Weekly（每周）、MoodAbove（心情高于）、SlackCount（摸鱼次数）、Idle（闲置）、Random（随机）、KpiOverload（绩效过载）、Annual（年度）、RarePool（稀有池）等。

事件字段：

字段	含义
Id	事件编号，如evt_01
Kpi/Mood/Insp	结算时三轴的变化量
ChainTo	后续连锁事件id，空则无
ChainChance	连锁概率0到1
FreezeSeconds	大于0时冻结画面演出
Kind	触发类型
Param/Param2	触发阈值
WageMultMin/Max	年度谜题工资倍率范围

部分事件定义：

Id	触发	效果
evt_01	Scripted	心情+10，5%连锁evt_01b
evt_02	Weekly	灵感+15，必连锁evt_02b
evt_03	MoodAbove 50	心情-10、灵感+20

evt_04	SlackCount 5	绩效-2、心情+15
evt_05	Idle 600	心情-5，连锁回正
evt_07	KpiOverload 90	冻结30秒演出
evt_09	Annual	工资倍率0到10倍谜题
evt_12	StageKpiMet	绩效+30、心情+20

节奏锚点：

常量	值	含义
EventAvgIntervalSec	20	平均20秒一张卡
EventCooldownSec	1800	同一事件30分钟冷却
FxBoundKpi	30	单次绩效变化上限
FxBoundMood	20	单次心情变化上限
FxBoundInsp	20	单次灵感变化上限

事件结算后三轴变化被限制在 FxBound 内，防止一张卡把数值打穿。带连锁的事件按 ChainChance 掷出后续卡。

9.1 事件流引擎

事件判定和时序由 WorkerEventFlow 跑，整个流程是 tick 流的纯函数，两次相同的运行结算出字节一致的卡，不使用随机API。构造时按事件的 Kind 把冻结表分类进不同容器：开场、条件、池、每周、年度、技能、阶段门，新增或调数值不需要改引擎、只需重跑签名。

开场门控：脚本化的开场派会（evt_01，开局约10秒）独占第一张卡位，是首个笑点节拍，其他任何触发都不允许在它之前弹，保证玩家先看到一张卡。

阶段范围池：Random、StageRandom、RarePool
三类走确定性轮转，索引自增对池数量取模，每240秒一个槽，落在180到300秒的节奏带内。

连锁时序：必连锁卡在主卡后4秒落地，概率连锁卡在8秒落地。像0.05这种连锁概率不靠随机，而是在结算计数器上走确定性的20分之一槽。

节奏约束：非连锁卡之间保持20秒最小间隔，同一id卡保持30分钟冷却。FlowState 是触发条件每帧读取的模拟快照，字段含增量时间、模拟时间、三轴、职级、摸鱼次数、闲置秒数。结算产出 ResolvedEvent，带三轴变化和冻结时长，交给宿主和模拟应答。

绩效过载事件（evt_07）要求过载状态持续5秒才触发冻结演出，避免瞬时抖动误触发。经理阶段（职级索引3）才开放对应的阶段门事件。

9.2 事件判定流程

事件流每帧推进的处理顺序：

1. 读取当前 FlowState 快照
2. 若开场未完成，只判开场卡，完成后放行其他触发
3. 检查条件类事件（心情阈值、摸鱼次数、闲置、过载）
4. 到达池节奏点时从轮转池取下一张
5. 校验节奏间隔和同id冷却，不满足则跳过
6. 结算三轴变化并按 FxBound 裁剪
7. 需要冻结的卡进入冻结演出
8. 带连锁的卡按规则排入待发连锁队列，到点落地

10 晋升门槛

WorkerRankGate 判定能不能升下一职级。晋升有四道门槛，阈值在GateKpiThreshold，需要绩效、技能、标记事件同时满足。标记事件 GateMarkerEventId 要求玩家经历过指定事件（如首次汇报、首次摸鱼被抓），光攒数值不够，得走过流程。

满足门槛后 WorkerCareer 执行晋升，RankIndex 加一，工资按新职级系数算。每道门槛对应一个标记事件，避免纯数值堆料跳过玩法。

11 投胎重开 NG+

玩家干到CEO或主动退休后进入投胎重开。退休时结算：保留一部分工资（coin-keep律），重置职级和状态轴，ngLoop加一。

重开后工资带继承倍率，每多一周目乘 (1 + InheritWageMultPerLoop × ngLoop)，越重开起点越高。retireCount 记退休次数，跨周目累计。

12 离线收益

WorkerOffline 处理离线。玩家回来时按离线时长算收益：

```
double OfflineGainHours(double awayHours, bool weekend,
int rank, int pptLv, int manageLv, int ngLoop)
{
double span = Min(awayHours, MaxOfflineHours);
double rate = WagePerSec(rank, pptLv, ngLoop);
double manage = SkillMult(manageLv, OfflineBonus);
return span * 3600.0 * rate * manage
* (weekend ? WeekendOfflineMult : 1f);
}
```

离线时长最多算12小时，向上管理技能提升离线收益，周末再乘1.2。离线收益可看广告翻倍，翻倍次数受 OfflineDoublePerSpan 限制。

13 外观十连获取

WorkerGacha 是纯IAA的外观获取，不接真实付费，看一条激励视频换十连。

奖池11个外观，总权重100：

外观Id	档位	权重
skin_wrinkled_shirt	Common	10
skin_coffee_stain_tie	Common	10
skin_free_conf_shirt	Common	10
badge_plastic_lanyard	Common	10
badge_carpool_sticker	Common	10
skin_short_sleeve_suit	Common	10
skin_gym_bro_fit	Rare	10
skin_silent_spec_auth	Rare	10
badge_silver_lanyard	Rare	10
skin_ceo_golden_suit	Epic	5
badge_penthouse_keycard	Epic	5

普通权重合计60、稀有30、史诗10。

获取随机走 splitmix64 整数链，是 (seed, pull index) 的纯函数，不用System.Random，相同种子产出完全一致的序列。

保底：一次十连里前9次都没出史诗，第10个槽位强制升级成史诗，每条广告都带保底，不出空十连。GachaState 记总抽数、距上次史诗抽数、史诗总数，由存档层持久化。

广告门控：外观获取广告每日10次，走共享 AdCapPolicy，先查类别门控、再拿广告结果、最后消耗本地次数；广告失败时本次获取可重试、状态不变。

14 存档

WorkerSaveData 是本地职业存档的镜像，纯本地、无服务器。字段：

字段	类型	含义
schemaVersion	int	存档版本，当前1
coin	double	钱包，投胎后保留
exp	double	经验
rankIndex	int	职级
ngLoop	int	投胎周目数
retireCount	int	退休次数
kpi/mood/insp	float	三条状态轴
working	bool	工作/摸鱼状态
simTime	double	模拟时间确定性载体

wageBeatCarry	double	工资节拍进位载体
skillLevels	int[4]	四分支技能等级
eventsSeen	string[]	见过的事件id集合
offlineUnix	long	离线时间戳

存档用 JsonUtility 序列化成JSON。Capture 从活着的模拟抓快照，四个技能等级按分支顺序写入，eventsSeen 从事件日志去重折叠。

加载律：损坏、空、版本不符的JSON拒绝加载，返回全新开局面孔，不崩溃、不部分应用：

```
static WorkerSaveData FromJsonOrFresh(string json)
{
if (!string.IsNullOrEmpty(json))
{
try {
var save = JsonUtility.FromJson<WorkerSaveData>(json);
if (save != null && save.schemaVersion == MirrorSchemaVersion
&& save.skillLevels != null
&& save.skillLevels.Length == 4)
return save;
} catch (Exception) { /*         */ }
}
return FreshStart();
}
```

simTime 和 wageBeatCarry 两个确定性载体随存档走，恢复后接着原来的节拍和事件流继续，不会因为重登打乱工资节奏。

15 主宿主

WorkerMainHost 是 MonoBehaviour，负责驱动纯C#模拟：每帧拿 deltaTime 调 WorkerSim.Tick，把模拟结果刷到HUD，处理按钮输入，弹窗时暂停模拟。宿主还管 PlayerPrefs 里的外观获取和职业页状态，通过只读加载器和数据层合并，数据层本身不写 PlayerPrefs。

15.1 每日任务与徽章

WorkerDailyAch 管每日任务板，每个本地日历日发3个任务。每日任务三选是日期键通过 FNV-1a 加 splitmix64 链的纯函数，不用随机API，同一天永远发同样三个任务、换一天发新的三个。

任务读取7种职业统计：摸鱼次数、事件卡、技能升级、工资节拍、晋升、史诗、退休。任务池冻结9行，每天发3个不重复的，目标按一次游玩时长调，工资节拍每秒1个、事件卡约20秒一张，所以高阶任务要求几分钟的真实游玩。部分任务：

统计	目标	金币	经验
摸鱼	3	150	0

摸鱼	10	400	60
事件	3	200	40
事件	8	500	120
技能	1	300	80
技能	3	900	200
工资节拍	120	250	0
工资节拍	600	800	0
晋升	1	1000	200

任务进度只算当天，每日板保留当日基线快照，不读终身总数。跨日重开时重置统计基线、清领取掩码，当天奖励每天可领一次；重开同一天保留所有领取和基线。奖励通过真实钱包和经验通道入账，被拒路径不改状态。

发牌从日期哈希里取三段、映射成三个互不重复的任务池索引，无偏抽样、同一天结果恒定：

```
QuestDef[] QuestsForDay(string dayKey)
{
ulong h = HashDay(dayKey);
int n = QuestPool.Length;
int a = (int)(h % (ulong)n);
int b = (int)((h >> 21) % (ulong)(n - 1));
if (b >= a) b++;
int lo = a < b ? a : b;
int hi = a < b ? b : a;
int c = (int)((h >> 42) % (ulong)(n - 2));
if (c >= lo) c++;
if (c >= hi) c++;
return new[] { QuestPool[a], QuestPool[b], QuestPool[c] };
}
```

领取是每天一次，先查领取掩码、再查进度，未完成或已领都原样返回、零状态改动，通过才经钱包和经验通道入账：

```
ClaimResult TryClaim(DailyBoard board, CareerLedger ledger, int qi, WorkerSim sim)
{
var q = QuestsForDay(board.DayKey)[qi];
if ((board.ClaimedMask & (1 << qi)) != 0) return AlreadyClaimed;
if (Progress(board, ledger, q.Stat) < q.Target) return Incomplete;
if (sim != null) { sim.DepositCoin(q.RewardCoin); sim.GrantExp(q.RewardExp); }
board.ClaimedMask |= 1 << qi;
return Granted;
}
```

进度是终身账本减当日基线、负值归零，跨日中途刷新也不会污染新板。

职场梗徽章族（如带薪拉屎大师、摸鱼宗师）按终身账本做纯阈值折叠，每个徽章一位、幂等单调、一旦获得不会取消。

16 广告

纯IAA，广告位：离线收益翻倍、外观获取、事件卡正面选项等，全部可选，核心进度不依赖广告。有每日次数上限（AdDailyCap）和冷却，广告预加载，失败时给跳过选项不卡流程。

17 容错与性能

- 存档解析失败用备份或全新开局，不丢主流程
- 资源缺失用占位，不白屏
- 全局异常捕获，广告失败不卡死
- 对象池复用事件卡和飘字
- 纯C#模拟核心可离线单测，数值表冻结后生成指纹
- 静态合批、按需加载，包体控制在20MB内

数值表冻结后通过 DeriveSigA 和 DeriveSigB 两条路径独立推导签名，两条路径结果一致才放行，任何数值被改动都会让指纹翻转，防止平衡表被误改。

模拟核心不依赖 Unity 的 Update 时序，所有结算都吃外部传入的 deltaTime，同一串输入在任何机器、任何帧率下跑出同样结果。这样做一是方便在编辑器外直接对核心跑断言，二是避免掉帧时工资和事件节奏被拉乱，帧率高低只影响画面、不影响数值。

模块之间靠事件回调通信，模拟层不持有任何 UI 引用，界面订阅自己关心的事件、自己决定怎么画。模拟跑崩了不会拖垮界面，界面临时卡一下也不会让模拟重复结算。存档、模拟、表现三层边界划清楚后，后期加玩法只在对应层动、不会牵一发动全身。

离线、广告、每日任务这些入口最终都回到模拟的存款和经验方法，钱和经验只有一个出处，不会出现某个渠道多发、账实不符的情况。

18 版本信息

项	内容
软件全称	摸鱼也升职游戏软件
软件简称	摸鱼也升职
版本号	V1.0
开发完成日期	2026年09月30日
发表状态	未发表
开发方式	独立开发
权利取得方式	原始取得
权利范围	全部权利

编程语言	C#
源程序量	约5400行（运行时代码）